15-418: Project Proposal

Parallelizing a quantum computation simulator

Ayush Garg

November 9, 2026

Project URL: *To be added after GitHub Pages deployment*

1 Summary

My goal is to parallelize the simulation of a quantum computer using GPUs, with the main focus on speeding up state-vector updates and scheduling the necessary computations as effectively as possible. I plan to implement the simulator from scratch in C++ and CUDA and evaluate it on the GHC machines.

2 Background

At its core, state-vector simulation of quantum circuits involves keeping track of a complex probability amplitude for every possible arrangement of bits. The squared magnitude of an amplitude gives the probability of observing that bit string. For example, a three-qubit system has the state.

$$|\psi\rangle = \alpha_0|000\rangle + \alpha_1|001\rangle + \alpha_2|010\rangle + \alpha_3|011\rangle + \alpha_4|100\rangle + \alpha_5|101\rangle + \alpha_6|110\rangle + \alpha_7|111\rangle, \qquad \sum_{i=0}^{7} |\alpha_i|^2 = 1.$$

In general, this means maintaining an array of 2^n complex numbers for n qubits. The memory requirement grows exponentially. Using two 64-bit floating-point values per amplitude, a 30-qubit state vector alone requires 16 GiB. Initially, I will use the known computational basis state $|0\cdots0\rangle$ and apply a sequence of quantum gates to evolve it.

In a Boolean circuit, NAND is a universal gate. Similarly, the gate set $\{H, T, \text{CNOT}\}$ can approximate any unitary quantum circuit to arbitrary accuracy. Note that since the outputs are continuous no finite set of gates can perfectly model a quantum computation so we have to settle for approximation.

2.1 The three gates

Hadamard (H) mixes the amplitudes of $|0\rangle$ and $|1\rangle$:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Phase (T) leaves the $|0\rangle$ amplitude unchanged and rotates the phase of the $|1\rangle$ amplitude:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \qquad T\frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{|0\rangle + e^{i\pi/4}|1\rangle}{\sqrt{2}}.$$

Controlled NOT (CNOT) flips a target bit when its control bit is 1. In control–target order,

$$|00\rangle \mapsto |00\rangle, \quad |01\rangle \mapsto |01\rangle, \quad |10\rangle \mapsto |11\rangle, \quad |11\rangle \mapsto |10\rangle.$$

For example, CNOT maps $(|00\rangle + |10\rangle)/\sqrt{2}$ to the entangled state $(|00\rangle + |11\rangle)/\sqrt{2}$.

3 Opportunities for parallelism

1. **Parallel state-vector updates.** A single-qubit gate can update 2^{n-1} disjoint amplitude pairs in parallel. Since the state space can be enormous, assigning these updates to GPU threads offers substantial parallel work.
2. **Independent qubit groups.** Groups that start unentangled and do not interact can be simulated separately. For example, starting from $|0000\rangle$, one worker can apply $H(q_0)$ followed by $\mathrm{CNOT}(q_0, q_1)$, while another applies $H(q_2)$ followed by $\mathrm{CNOT}(q_2, q_3)$. Each worker produces a two-qubit Bell state. Their results can be combined by a tensor product when a joint state is needed.
3. **Multiple devices (stretch goal).** Distributing the state vector across devices increases the available memory. However, gates that combine amplitudes on different devices require communication. I may not have time to implement this extension.

4 The challenge

There are many challenges with simulating a quantum computer.

The first and foremost is dealing with the memory requirements. An n-qubit quantum circuit requires maintaining 2^n complex amplitudes, so both memory requirements grow exponentially. These large state vectors exceed CPU caches and may exceed GPU memory. This will require me to be very mindful about the memory transfers done during the computation. Furthermore, splitting a state vector across devices or machines increases available memory, but gates that mix amplitudes in different partitions require communication, which can become a bottleneck for our code.

Another large challenge will be dealing with the workload's memory access pattern. For example, a general single-qubit gate updates pairs of amplitudes whose indices differ by 2^k, where k is the target qubit's bit position. This produces predictable access patterns, but the spacing between paired amplitudes varies substantially. Scheduling this effectively while being mindful of the threads cache locality seems like a worthy problem. I We must be very diligent about the memory access pattern since update performs relatively little arithmetic compared with the data it reads and writes, so memory bandwidth may limit speedup before all available compute resources are

Finally, this challenge will be difficult just because of how poorly quantum computer simulation scales. Trying to keep up with the exponential scaling will be incredibly difficult and our speedups will only take us so far. Trying to extract as many bits as possible out of the simulation seems like a fun and worthy when every additional bit makes our memory problems twice as painful and increases the work by at least a factor of 2.

5 Resources

I plan to use the GHC machines with NVIDIA GPUs for most of the computation. I will start from scratch and use Qiskit's CPU State vector implementation as a correctness reference. I do not hope to match the speed of a mature simulator as it also has GPU implementations and likely does much more complex optimizations, but I still hope to get a good speedup.

6 Goals and deliverables

6.1 Plan to achieve

1. **Sequential baseline.** Create a correct C++ state-vector simulator supporting H, T, and CNOT, together with reproducible benchmark circuits. Validate complex amplitudes against Qiskit on small circuits.
2. **GPU implementation.** Implement CUDA kernels for parallel amplitude updates and measure their speedup relative to the sequential implementation on identical circuits and precision.
3. **Scheduling and optimization.** Explore gate/kernel fusion and parallel evaluation of independent qubit groups where the circuit permits it. Evaluate how circuit structure and target-qubit positions affect performance.
4. **Application demonstration.** If feasible, use the simulator for a small Shor factoring demonstration, targeting a number such as 119. This requires a suitable arithmetic circuit and decomposition of its gates; intermediate measurement and reset may be needed depending on the construction [3].

My performance target is 100–250 over my single-threaded CPU baseline on sufficiently large workloads. While there are a large number of independent amplitude updates, I worry that memory bandwidth and launch overhead may prevent me from reaching this goal. I also hope that with this speedup I am able to simulate larger quantum circuits that were unfeasible just on the CPU.

To evaluate speedup I will have some diverse workload where we evaluate how the program scales across both the number of qbits and the length of the circuit. In general I don't plan to optimize heavily for very niche circuits like those with minimal entanglement.

6.2 Hope to achieve

The stretch goal is to distribute one state vector across multiple devices to increase the available memory. Even 30 qubits require 16 GiB in double precision, so additional devices may allow a few more qubits. As a note this might not be worth the effort due to the exponential requirements of the algorithm. For example even scaling to 4 different machines would only allow for 2 extra qubits.

6.3 Reduced-scope fallback

If progress is slower than expected, then I will ensure that I have a correct serial and single-GPU implementations. I might not be able to create as effective schedulers and I think that I will likely just end up with a lower speedup.

7 Platform choice

I plan to use C++ for the serial implementation and CUDA for GPU kernels on the GHC machines. I am choosing CUDA since we have worked with it in class and it maps naturally to the many independent amplitude updates required by each gate. When possible I will also use thrust to make the code easier to read and faster.

8 Schedule

Period	Planned work and checkpoint
November 9–16	Create the serial simulator, gate representation, correctness checks, and initial benchmark circuits.
November 16–23	Complete basic CUDA kernels for parallel state-vector updates. Validate against the serial implementation and collect initial timings.
November 23–30	Explore independent-group evaluation and gate/kernel fusion.
November 30–December 7	Optimize kernels, run benchmark sweeps,investigate bottlenecks, and prepare the report. Decide if time is better spent attempting multi-device execution only if that seems to be the only way to further scale the project.
December 7–11	Finish validation, write the final report, and prepare the poster.